

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE LABORATORIO

(formato estudiante)

INFORMACIÓN BÁSICA					
ASIGNATURA:	Computación Gráfica, Visión Computacional y Multimedia				
TÍTULO DE LA PRÁCTICA:	<i>Clasificación y Reconocimiento</i>				
NÚMERO DE PRÁCTICA:	<i>09</i>	AÑO LECTIVO:	<i>2026A</i>	NRO. SEMESTRE:	<i>IX</i>
FECHA DE PRESENTACIÓN	<i>07/07/2026</i>	HORA DE PRESENTACIÓN	<i>23:00</i>		
INTEGRANTE (s): Vera Zapacayo Jhoer Luis				NOTA:	
DOCENTE(s): <i>Ing. Nieto Valencia Rene Alonso</i>					

SOLUCIÓN Y RESULTADOS
I. SOLUCIÓN DE EJERCICIOS/PROBLEMAS 1. Roboflow y YOLOv8 Descripción: Roboflow es una plataforma todo en uno que facilita y mejora el desarrollo de proyectos de visión por computadora. Se utiliza en tareas como la detección de objetos, la clasificación de imágenes y la segmentación. Es una herramienta muy conocida por su facilidad para organizar conjuntos de datos y preparar modelos de visión artificial. Roboflow se combina frecuentemente con frameworks como YOLOv8, ya que permite gestionar y preprocesar los datos de manera rápida y eficiente, lo que acelera el entrenamiento de modelos de detección de objetos. Características de Roboflow para detección de objetos • Administración de datasets: Roboflow facilita la carga, organización y etiquetado de imágenes. Ofrece una interfaz visual intuitiva que permite hacer anotaciones manualmente o importar conjuntos de datos previamente anotados desde otras herramientas.

- **Procesamiento previo inteligente:** Proporciona múltiples opciones para mejorar tus datos antes del entrenamiento, como aumentos automáticos (data augmentation), cambios de escala, giros, recortes, entre otras transformaciones útiles.
- **Compatibilidad con múltiples frameworks:** Permite exportar tus datasets anotados en formatos listos para usar con frameworks de visión como YOLOv8 (una función que se puede probar más adelante).
- **Entrenamiento y modelos integrados:** Cuenta con la opción de entrenar modelos directamente en la nube desde la misma plataforma o, si se prefiere, exportar los datos para entrenarlos localmente. También ofrece modelos ya entrenados que pueden aplicarse a tareas comunes.
- **Evaluación del rendimiento del modelo:** Incluye herramientas para analizar qué tan bien funciona tu modelo, proporcionando métricas como precisión (accuracy), recall y otras medidas esenciales para entender su efectividad.

Características de YOLOv8 (You Only Look Once v8)

- **Arquitectura mejorada:** YOLOv8 introduce mejoras significativas en su diseño interno, lo que le permite detectar objetos pequeños con mayor precisión y mejorar el rendimiento general del modelo.
- **Soporte para múltiples tareas:** Además de la detección de objetos, esta versión también es capaz de realizar clasificación de imágenes y segmentación de instancias, lo que la convierte en una herramienta flexible para diferentes casos de uso.
- **Modelos ya entrenados:** Ofrece modelos previamente entrenados que se pueden adaptar a necesidades específicas mediante técnicas de aprendizaje por transferencia (transfer learning).
- **Rendimiento en tiempo real:** Está diseñada para funcionar de manera rápida y eficiente, lo que la hace ideal para aplicaciones en vivo, como cámaras de vigilancia, drones o dispositivos conectados (IoT).
- **Integración sencilla:** YOLOv8 funciona bien con bibliotecas populares como PyTorch, y se puede conectar fácilmente con plataformas como Roboflow para un flujo de trabajo más ágil.

Empezamos creando un nuevo proyecto, en este especificamos el grupo de anotación que queremos reconocer y el tipo de proyecto que deseamos, para esto especificaremos la detección de objetos.



Clasificación y Reconocimiento > [New Public Project](#)

Project Name

Clasificación_Reconocimiento

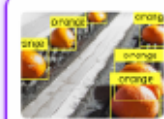
License

CC BY 4.0

Annotation Group

dogs

Project Type



Object Detection

Identify objects and their positions with bounding boxes.

Best For [Counting](#) [Tracking](#)



Classification

Assign labels to the entire image.

☒ Single-Label ☐ Multi-Label

Best For [Filtering](#) [Content Moderation](#)



Instance Segmentation



Keypoint Detection

Una vez creado el proyecto se procederá a subir las imágenes que utilizaremos para entrenar el modelo de reconocimiento. En este caso, el objetivo es que el sistema sea capaz de identificar imágenes de perros. Para este proyecto, se utilizaron 20 imágenes de perros como conjunto de datos inicial para el entrenamiento del modelo.

Drag and drop images, annotations, and videos.

[.jpg, .png, .webp, .gif](#) [In 16 formats](#) [.mov, .mp4](#)
 Max size of 20MB and 10,240 pixels per dimension.

Select Files

Select Folder

Save and Continue →



1.jpg



2.jpg



3.jpg



4.jpg



5.jpg



6.jpg



7.jpg



8.jpg



9.jpg



10.jpg



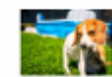
11.jpg



12.jpg



13.jpg



14.jpg



15.jpg



16.jpg



17.jpg



18.jpg

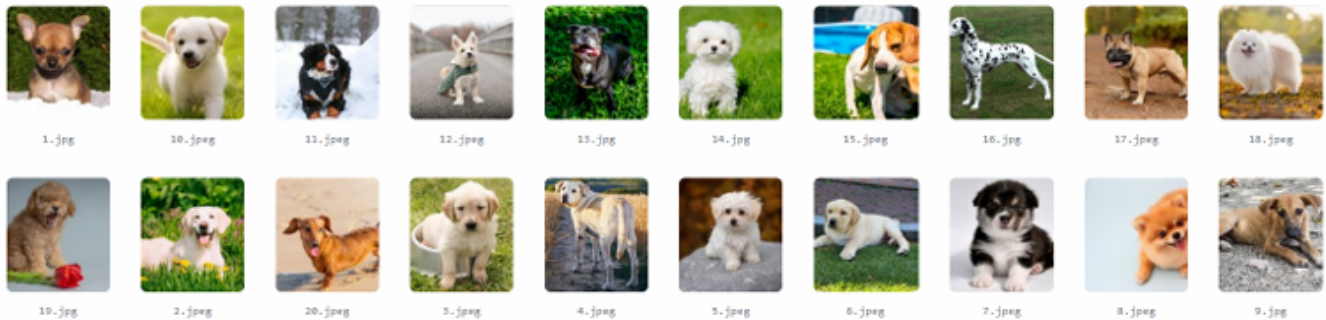


19.jpg

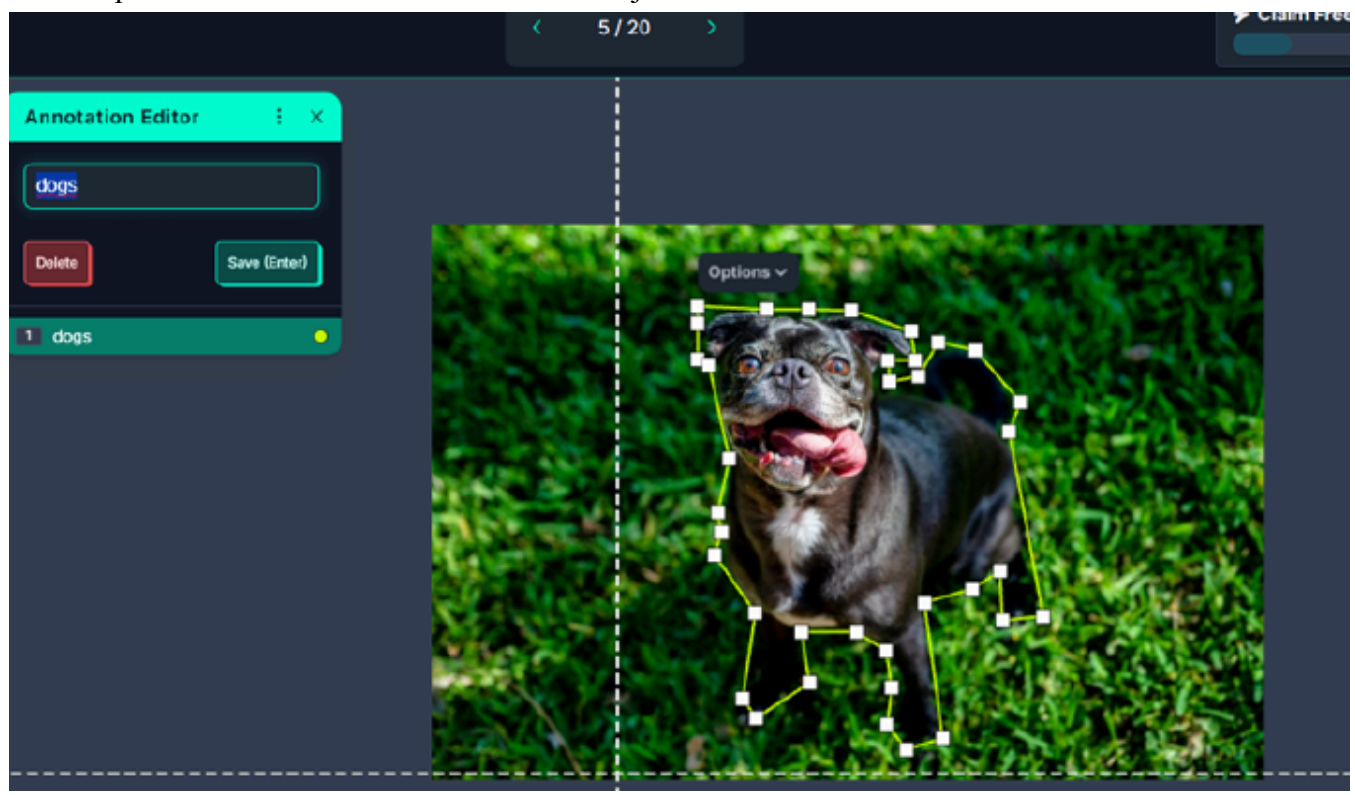


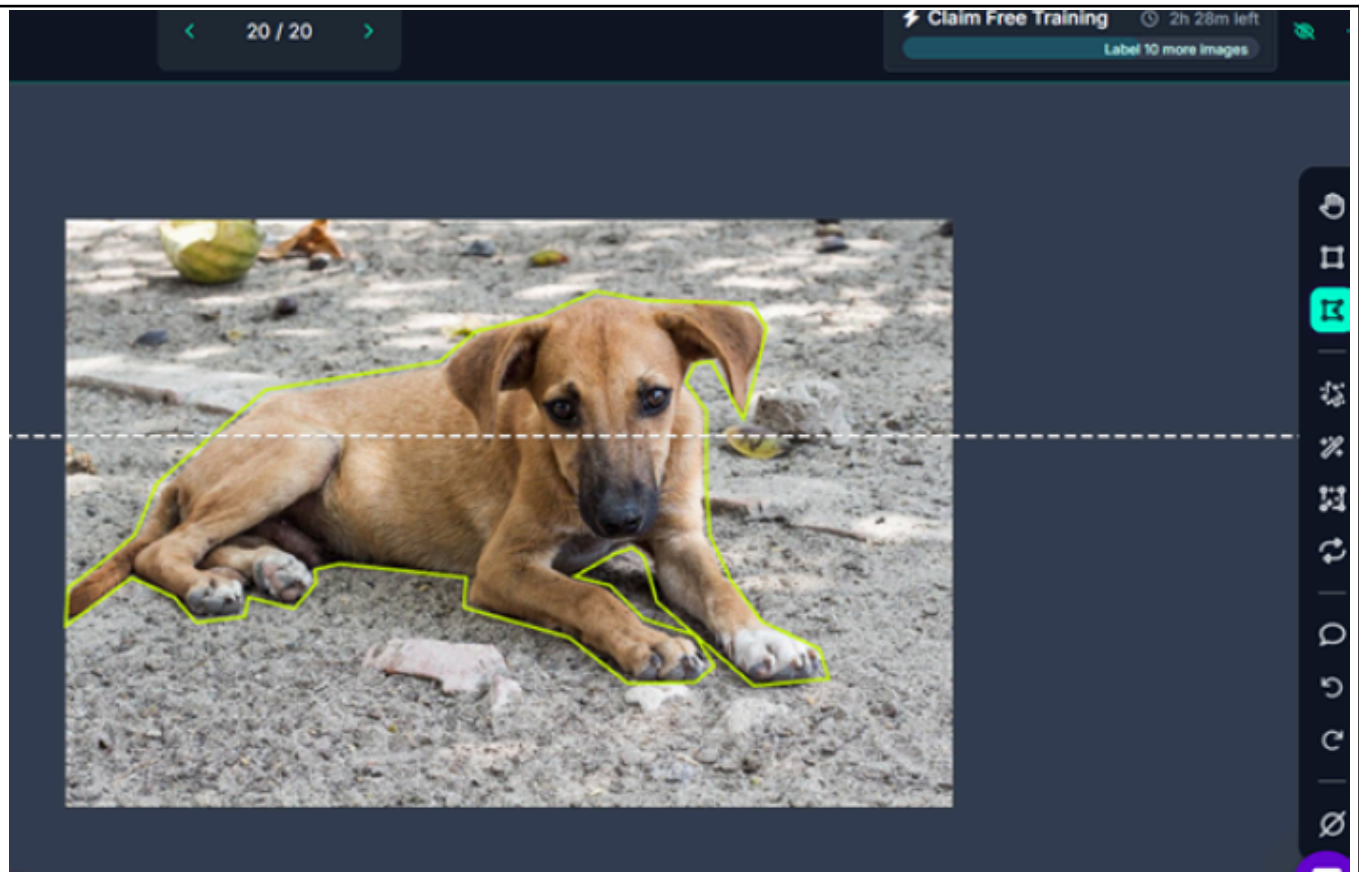
20.jpg

Después de cargar las imágenes que se utilizarán en el proyecto, se debe hacer clic en el botón ‘Annotate images’ para comenzar con el proceso de etiquetado o anotación, el cual es para que el modelo pueda aprender a reconocer correctamente los objetos presentes en las imágenes.



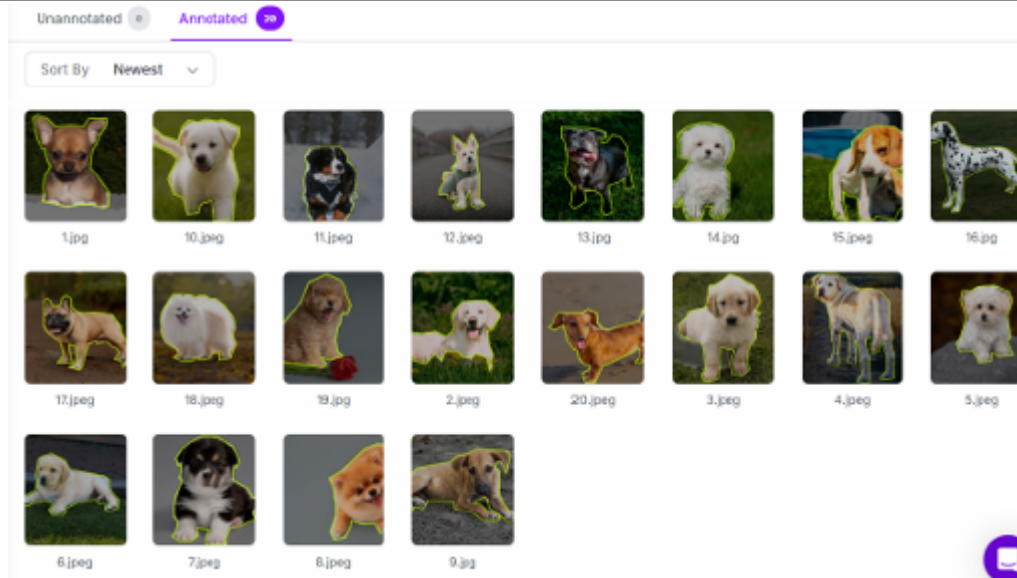
Al hacer clic en ‘Annotate images’, se abrirá el editor de anotaciones, una herramienta donde marcaremos manualmente la ubicación de los perros en cada imagen. Este proceso debe repetirse para todas las imágenes cargadas en el proyecto, ya que es fundamental para que el modelo aprenda a reconocer correctamente los objetos durante el entrenamiento.





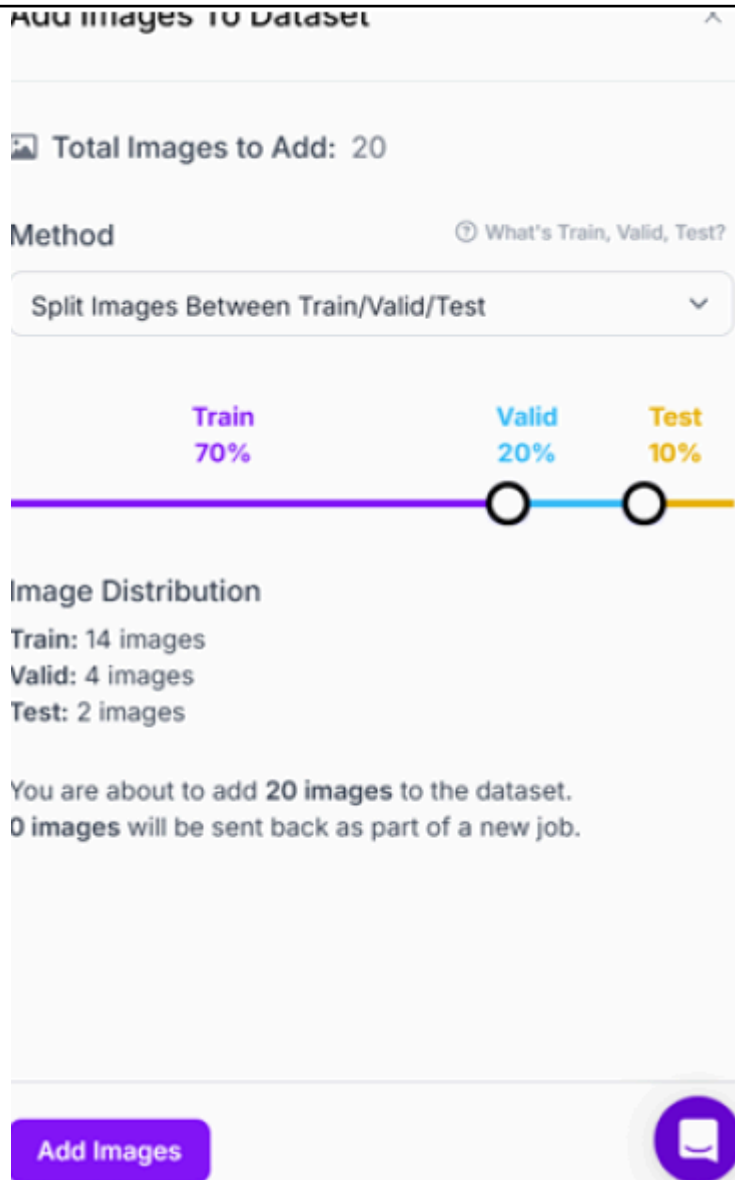
Una vez completada la anotación de los perros en las 20 imágenes, utilizamos el botón de retroceso para volver a la pantalla anterior. Allí podremos ver las imágenes ya marcadas con las etiquetas correspondientes para la detección de los animales.

A continuación, hacemos clic en el botón 'Add 20 images to Dataset', lo cual añadirá estas imágenes anotadas al conjunto de datos del proyecto. Después de esto, se habilitarán otras opciones que permiten aplicar técnicas de aumento de datos (data augmentation), generando imágenes adicionales a partir de las ya existentes, si así lo deseamos.



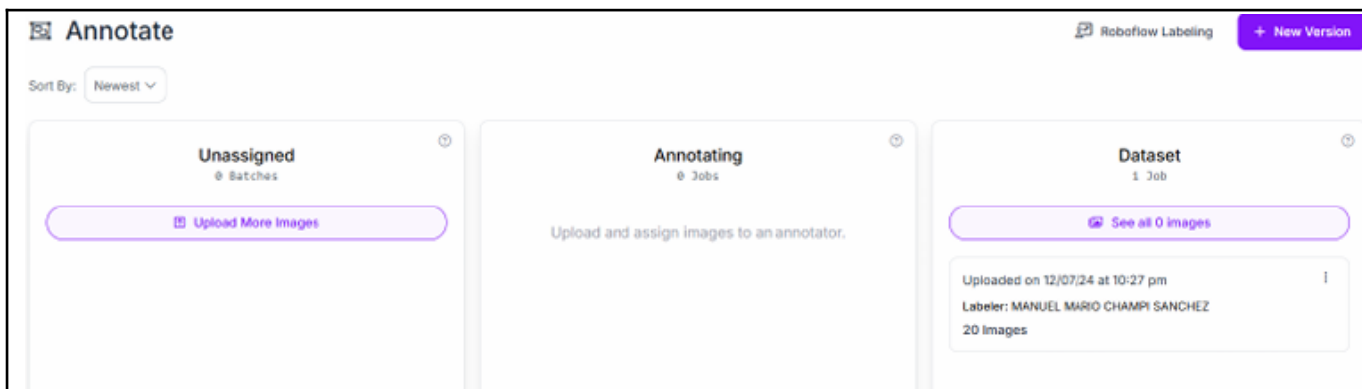
A continuación, se mostrará la distribución del conjunto de datos, dividido en tres partes: entrenamiento (train), validación (valid) y prueba (test). Esta separación es importante para evaluar el rendimiento del modelo de forma precisa.

Luego, hacemos clic en el botón ‘Add images’ para confirmar y agregar las imágenes al dataset final con el que se trabajará en el entrenamiento del modelo.

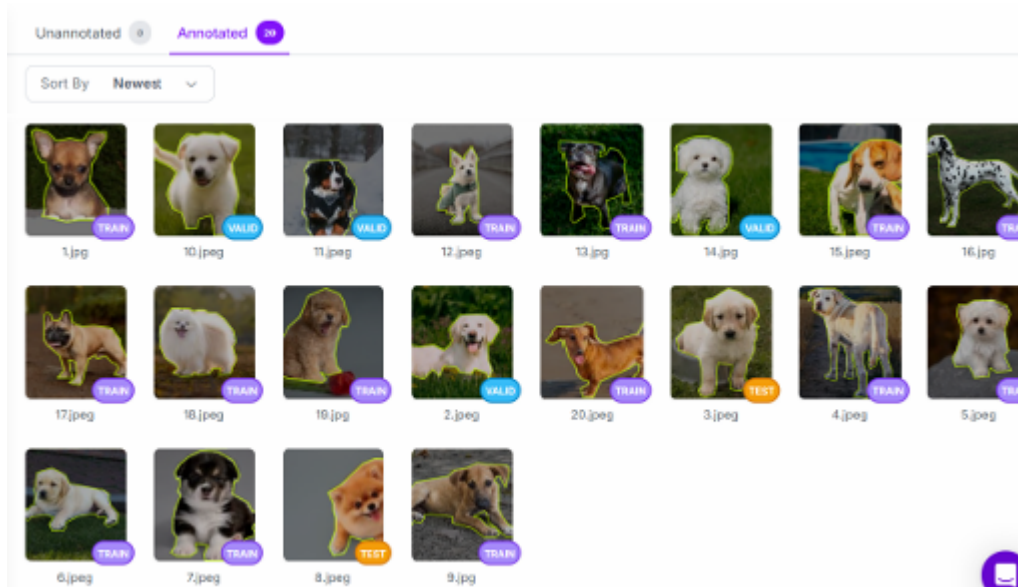


Después de agregar las imágenes al conjunto de datos, se abrirá una nueva ventana con varias funciones disponibles. De todas ellas, la que nos interesa es la sección llamada 'Dataset', donde se encuentran nuestras imágenes ya anotadas y listas para su análisis, enfocadas en la detección de perros.

En esta sección, haremos clic sobre la pequeña tarjeta o recuadro que muestra el nombre del autor y la cantidad de imágenes cargadas, para acceder directamente al conjunto de datos que usaremos en el entrenamiento.



Al hacer clic en el recuadro del dataset, se nos mostrará nuevamente el conjunto de imágenes anotadas. En la parte inferior derecha de cada imagen, aparecerá un pequeño resaltado o etiqueta que indica a qué parte de la distribución pertenece: entrenamiento (train), validación (valid) o prueba (test), tal como se mostró en la distribución anterior.



Para continuar con el proceso, nos dirigimos a la pestaña 'Annotation' y creamos una nueva versión del conjunto de datos. En esta etapa, dejaremos la configuración del procesador por defecto, sin realizar modificaciones, y simplemente continuamos con los pasos siguientes.

3

Preprocessing

What can preprocessing do?

Decrease training time and increase performance by applying image transformations to all images in this dataset.

Auto-Orient

Edit

×

Resize

Stretch to 640×640

Edit

×



Add Preprocessing Step

Continue

4

Augmentation

En la siguiente etapa aplicaremos dos técnicas de aumento de datos (data augmentation) para generar imágenes adicionales a partir de las existentes. Primero, utilizaremos un flip horizontal que invertirá las imágenes, creando nuevas versiones reflejadas. Luego, aplicaremos una rotación con ángulos de $+15^\circ$ y -15° para obtener aún más variaciones.

En teoría, con estas transformaciones el número total de imágenes aumentaría hasta 80. Sin embargo, debido a que algunas imágenes no son lo suficientemente claras o estables para ser reconocidas correctamente por el programa, no todas las nuevas versiones se generan o se añaden al dataset. Como resultado, el conjunto final quedará con 47 imágenes en total.

4

Augmentation

What can augmentation do?

Create new training examples for your model to learn from by generating augmented versions of each image in your training set.

Flip

Horizontal

Edit

×

Rotation

Between -15° and $+15^\circ$

Edit

×



Add Augmentation Step

Continue

Clear All

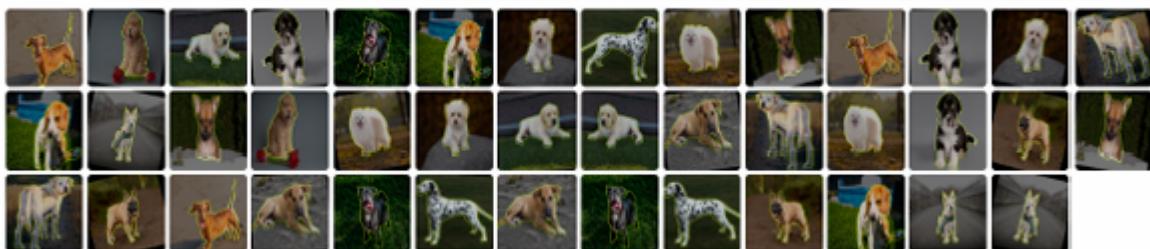
Imágenes totales generadas

47 Total Images

Train 41

Valid 4

Test 2



41 / 41 images

En la ventana de la versión del dataset, encontraremos la opción 'Download Dataset'. Al seleccionar esta opción, se nos proporcionará un código que podremos utilizar para entrenar nuestro modelo de reconocimiento de imágenes con la versión que hemos creado.

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 11

This version doesn't have a model.

Train an optimized, state of the art model with Roboflow or upload a custom trained model to use features like Label Assist and Model Evaluation and deployment options like our auto-scaling API and edge device support.

[Custom Train and Upload](#)
[Train with Roboflow](#)
Available Credits: 3

47 Total Images [View All Images →](#)



Dataset Split:

Dataset Split	Train Set	Valid Set	Test Set
	41 Images	4 Images	2 Images

Al hacer clic en el botón ‘Download Dataset’, aparecerá una ventana emergente que nos permitirá elegir el formato de exportación. En este caso, seleccionaremos el formato YOLOv8 y procederemos para continuar con la descarga.

Download
✕

Format

YOLOv8
▼

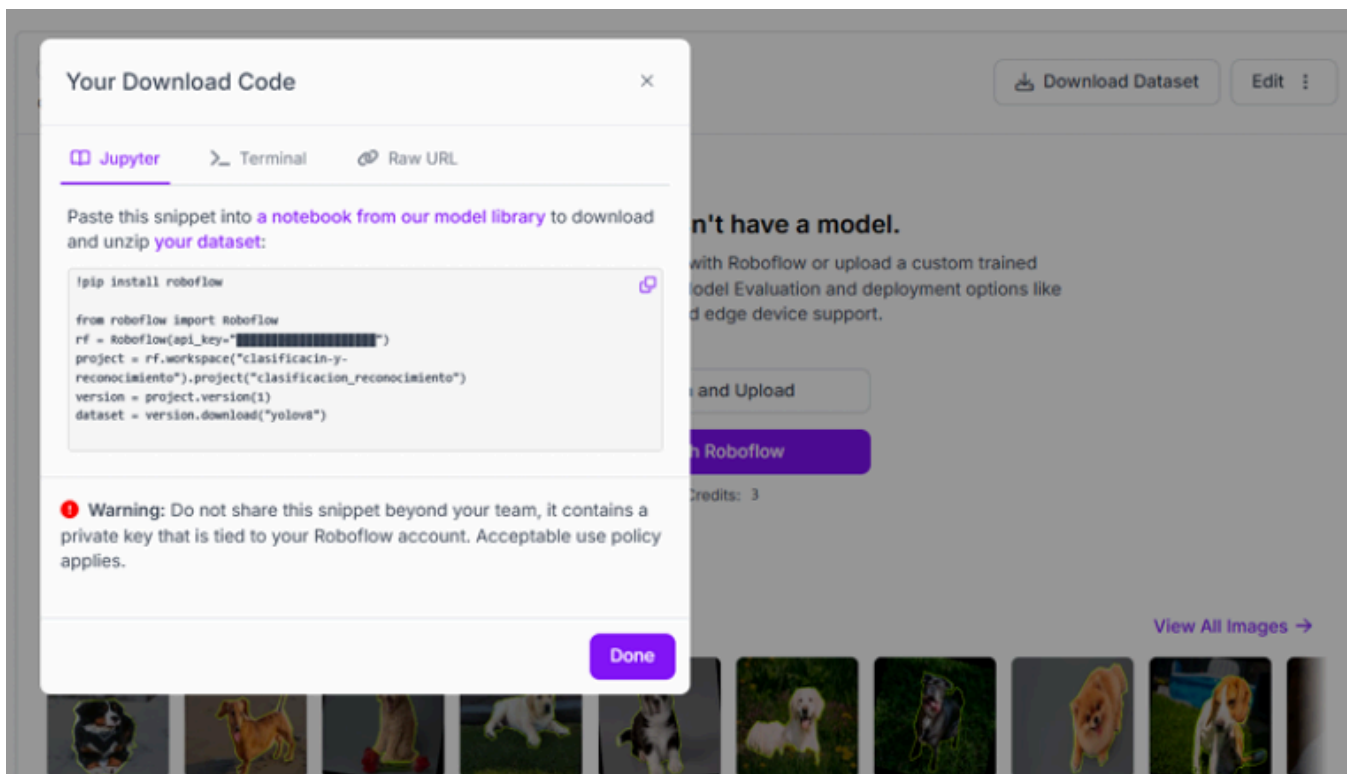
TXT annotations and YAML config used with YOLOv8.

☐ Download zip to computer
☒ Show download code

Cancel

Continue

A continuación, se mostrará una ventana pequeña que contiene un código en Python, adaptado al formato de YOLOv8, el cual podremos usar directamente para entrenar nuestro modelo con el dataset que hemos preparado.



Google Colab En google colab entrenaremos nuestro Dataset para el reconocimiento de las imágenes. Empezaremos declarando nuestra variable ‘HOME’ para cargar o generar archivos en la misma ventana.

```
[ ] import os
HOME = os.getcwd()
print (HOME)
```

 /content

Después, procederemos a descargar la librería ‘ultralytics’, que es fundamental para realizar tareas de detección de objetos, personas o animales. Dentro de esta librería se encuentra la implementación de la función YOLO, que utilizaremos para entrenar y ejecutar nuestro modelo.

```
!pip install ultralytics
from IPython import display
display.clear_output()

import ultralytics
ultralytics.checks()

Ultralytics 8.3.47 Python-3.10.12 torch-2.5.1+cu121 CPU (Intel Xeon 2.20GHz)
Setup complete (2 CPUs, 12.7 GB RAM, 32.7/107.7 GB disk)

[ ] from ultralytics import YOLO

from IPython.display import display, Image
```

Después, pegaremos el código que nos proporcionó Roboflow, añadiendo una variable llamada Home que indicará la ubicación donde se guardará el dataset. Al ejecutar el código, se descarga y descomprime el proyecto creado en Roboflow, el cual lleva por nombre 'Clasificación_Reconocimiento'.

```
!mkdir {HOME}/datasets
%cd {HOME}/datasets

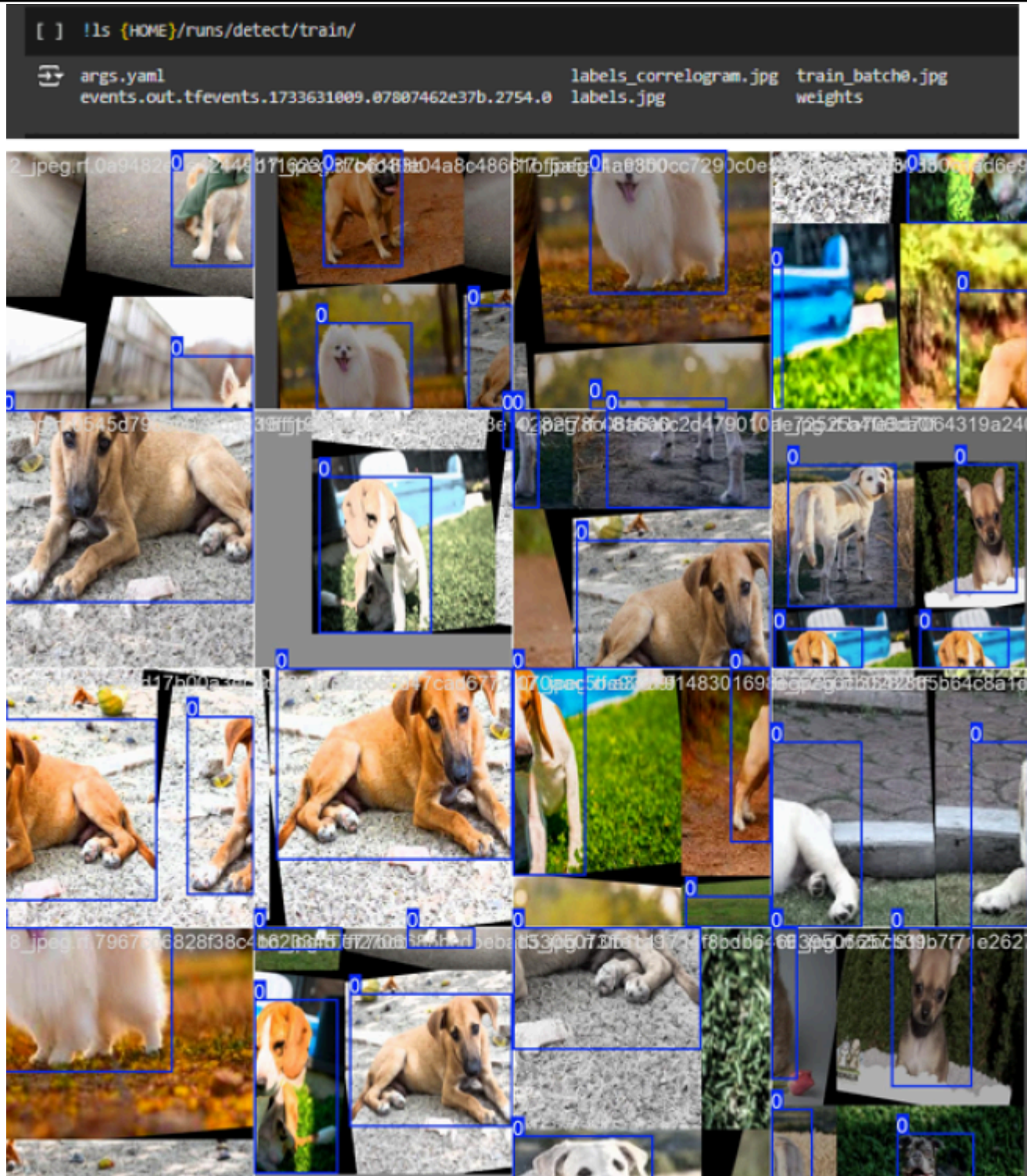
!pip install roboflow

from roboflow import Roboflow
rf = Roboflow(api_key="wrfu9v8GIHcDgh116awn")
project = rf.workspace("clasificacin-y-reconocimiento").project("clasificacion_reconocimiento")
version = project.version(1)
dataset = version.download("yolov8")

/content/datasets
Requirement already satisfied: roboflow in /usr/local/lib/python3.10/dist-packages (1.1.49)
Requirement already satisfied: certifi in /usr/local/lib/python3.10/dist-packages (from roboflow) (2024.8.30)
Requirement already satisfied: idna==3.7 in /usr/local/lib/python3.10/dist-packages (from roboflow) (3.7)
Requirement already satisfied: cyclur in /usr/local/lib/python3.10/dist-packages (from roboflow) (0.12.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.10/dist-packages (from roboflow) (1.4.7)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.10/dist-packages (from roboflow) (3.8.0)
Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.10/dist-packages (from roboflow) (1.26.4)
Requirement already satisfied: opencv-python-headless==4.10.0.84 in /usr/local/lib/python3.10/dist-packages (from roboflow) (4.10.0.84)
Requirement already satisfied: Pillow>=7.1.2 in /usr/local/lib/python3.10/dist-packages (from roboflow) (11.0.0)
Requirement already satisfied: python-dateutil in /usr/local/lib/python3.10/dist-packages (from roboflow) (2.8.2)
Requirement already satisfied: python-dotenv in /usr/local/lib/python3.10/dist-packages (from roboflow) (1.0.1)
Requirement already satisfied: requests in /usr/local/lib/python3.10/dist-packages (from roboflow) (2.32.3)
Requirement already satisfied: six in /usr/local/lib/python3.10/dist-packages (from roboflow) (1.16.0)
Requirement already satisfied: urllib3>=1.26.6 in /usr/local/lib/python3.10/dist-packages (from roboflow) (2.2.3)
Requirement already satisfied: tqdm>=4.41.0 in /usr/local/lib/python3.10/dist-packages (from roboflow) (4.66.6)
Requirement already satisfied: PyYAML>=5.3.1 in /usr/local/lib/python3.10/dist-packages (from roboflow) (6.0.2)
Requirement already satisfied: requests-toolbelt in /usr/local/lib/python3.10/dist-packages (from roboflow) (1.0.0)
Requirement already satisfied: filetype in /usr/local/lib/python3.10/dist-packages (from roboflow) (1.2.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib->roboflow) (1.3.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib->roboflow) (4.55.1)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib->roboflow) (24.2)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib->roboflow) (3.2.0)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.10/dist-packages (from requests->roboflow) (3.4.0)
loading Roboflow workspace...
loading Roboflow project...
Downloading Dataset Version Zip in Clasificación_Reconocimiento-1 to yolov8: 100%|██████████| 2413/2414 [00:00<00:00, 13983.74it/s]

Extracting Dataset Version Zip to Clasificación_Reconocimiento-1 in yolov8: 100%|██████████| 106/106 [00:00<00:00, 5358.52it/s]
```

Con la siguiente línea de código generamos las imágenes de salida donde el modelo realiza la detección de los perros. En estas imágenes se muestran los animales identificados junto con las etiquetas y recuadros delimitadores (bounding boxes) de color azul.



	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 16

Ventajas

- **Flujo de trabajo simplificado:** Roboflow ofrece un entorno intuitivo que abarca desde la gestión y anotación de datasets hasta la exportación de modelos, facilitando todo el proceso.
- **Compatibilidad con YOLOv8:** Permite generar datasets en formatos compatibles con YOLOv8, simplificando el entrenamiento de estos modelos.
- **Optimización de datos:** Incluye técnicas avanzadas de preprocesamiento y aumento de datos que mejoran la calidad del dataset y, por ende, el rendimiento del modelo.
- **Rendimiento mejorado:** YOLOv8 cuenta con una arquitectura optimizada que equilibra velocidad y precisión, ideal para aplicaciones en tiempo real.
- **Entrenamiento más eficiente:** Gracias a sus herramientas avanzadas de preprocesamiento y aumentos, YOLOv8 puede entrenarse más rápido y con menos datos.
- **Soporte comunitario activo:** Ambas plataformas cuentan con comunidades activas que proporcionan soporte, actualizaciones y mejoras constantes.

Desventajas

- **Dependencia de la nube (versión gratuita):** Muchas funciones avanzadas de Roboflow requieren conexión a internet y la versión gratuita limita el uso de recursos y tamaño de datasets, lo cual puede dificultar proyectos grandes.
- **Costo elevado para proyectos avanzados:** Las suscripciones premium pueden resultar costosas para pequeñas empresas o proyectos individuales que demanden un uso intensivo.
- **Compatibilidad limitada sin ajustes:** Aunque soporta varios formatos, puede ser necesario realizar configuraciones adicionales para integrarlo perfectamente con frameworks personalizados.
- **Carga en la preparación de datos:** El etiquetado y limpieza manual de grandes volúmenes de datos puede ser una tarea laboriosa y que consume mucho tiempo.
- **Compromiso entre velocidad y precisión:** YOLOv8 es rápido, pero en algunos casos específicos (objetos pequeños o densos) puede no alcanzar la precisión de modelos especializados.
- **Curva de aprendizaje técnica:** Configurar y entrenar YOLOv8 requiere conocimientos técnicos y manejo de herramientas como PyTorch y CUDA, lo que puede ser complejo para principiantes.
- **Requisitos de hardware altos:** Entrenar modelos desde cero demanda recursos computacionales potentes, como GPUs avanzadas, lo que puede limitar a quienes disponen solo de hardware básico.
- **Menos adecuado para casos muy específicos:** Para tareas como segmentación semántica o detección extremadamente precisa, frameworks especializados como Detectron2 podrían ser mejores opciones.

- **Falta de flexibilidad para aplicaciones muy personalizadas:** Para problemas muy particulares, YOLOv8 podría requerir modificaciones profundas que podrían afectar el rendimiento.

II. SOLUCIÓN DEL CUESTIONARIO

1. ¿Describan un algoritmo utilizado para el reconocimiento y clasificación de objetos?

Uno de los algoritmos más destacados para el reconocimiento y clasificación de objetos es YOLO (You Only Look Once). YOLO está basado en redes neuronales convolucionales (CNN) y está diseñado para detectar y clasificar objetos en imágenes o videos en tiempo real. A diferencia de otros métodos que analizan la imagen varias veces, YOLO realiza todas las predicciones (tanto la ubicación como la categoría de los objetos) en una sola pasada por la imagen, lo que lo hace extremadamente rápido y eficiente.

2. ¿Qué problemas pueden ocurrir al realizar el reconocimiento y clasificación de objetos?

- **Datos insuficientes o desequilibrados:** Falta de variedad en las imágenes o clases desbalanceadas afectan la precisión.
- **Errores en anotaciones:** Etiquetas incorrectas llevan a malas predicciones.
- **Problemas del modelo:** Sobreajuste o limitaciones para detectar objetos pequeños o muy juntos.
- **Condiciones ambientales:** Mala iluminación, objetos ocultos o ángulos raros dificultan la detección.
- **Requisitos computacionales:** Necesidad de hardware potente y posibles retrasos en tiempo real.
- **Confusión entre clases similares:** El modelo puede confundir objetos parecidos o desconocidos.

III. CONCLUSIONES

- Este laboratorio nos muestra el potencial de la inteligencia artificial para la detección de objetos, combinando Roboflow para gestionar y preparar datos, y Google Colab para entrenar y ejecutar el modelo YOLOv8. Gracias a Roboflow, se facilitó la anotación y exportación del dataset, mientras que Google Colab permitió entrenar el modelo en la nube de forma accesible.

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 18

RETROALIMENTACIÓN GENERAL

REFERENCIAS Y BIBLIOGRAFÍA

- [1] Gordon, V. S., & Clevenger, J. L. (2020). *Computer Graphics Programming in OpenGL with C++*. Stylus Publishing, LLC.
- [2] D. P. Kothari, G. Awari, D. Shrimankar , A. Bhende (2019), *Mathematics for Computer Graphics and Game Programming: A Self-Teaching Introduction*
- [3] Zoumana K. (2024). *Explicación de la detección de objetos YOLO*. Recuperado de: <https://www.datacamp.com/es/blog/yolo-object-detection-explained>
- [4] ultralytics. (2024) *Roboflow 100 Conjunto de datos*. Recuperado de: <https://docs.ultralytics.com/es/datasets/detect/roboflow-100/> [
- [5] Martin J. (2024). *Mi primer proyecto de AI con Yolo (Detección de Objetos en Tiempo Real)*. Recuperado de: <https://cdo-latam.com/mi-primer-proyecto-de-ai-con-yolo-deteccion-de-objetos-en-tiempo-real>